Nano-LLaVa: Optimizing Training Pipeline for Pre-trained Nanochat Extended to Image Captioning

Alexander Swartz, *Columbia Unversity*

I. Introduction

Background: The integration of visual modalities into Large Language Models (LLMs) typically introduces significant compute and memory overhead. To address the growing demand for models that can be trained with with consumer-grade hardware, this project implements a multimodal architecture following the lightweight LLaVA framework [1]. This involves leveraging both a pre-trained image encoder and pre-trained LLM and adding a minimal, trainable projection layer in-between. This project then focuses on optimizing the training pipeline to maximize training speed and throughput by profiling GPU utilization and identifying memory transfer bottlenecks.

Problem Statement: Given a pre-trained image encoder and LLM, the objective is find the minimal training setup needed to extend the LLM for image captioning. To ensure this new capability is efficient and accessible on consumer-grade hardware, the resulting training pipeline must meet the following performance goals on a single NVIDIA L4 GPU:

- Data Latency: $<$ 10% of total training time spent on DataLoader wait times.
- Transfer Efficiency: $<$ 5% of total training time spent on Host-to-Device (H2D) transfers.
- Hardware Saturation: $>$ 80% sustained GPU utilization.
- Inference Throughput: $>$ 5x speedup in inference tokens per second compared to the unoptimized baseline.

Scope: This workload of this project involved both augmenting the architecture of an existing LLM to support image captioning and optimizing the resulting pipeline for training and inference. The architectural modifications involved adding the linear project layer to map image embeddings into the LLM's vocabulary space and extending the Supervised Fine-Tuning (SFT) setup to support the image-captioning task. The optimization scope consists of experimenting with dataloader configurations, transitioning to offline image embeddings, Automatic Mixed Precision (AMP) memory optimization, and tuning inference throughput.

II. Literature Review

Nano-LLaVa mimics the first step of training in the foundational architecture introduced in the "Visual Instruction Tuning" [1] study called LLaVa (Large Language-and-Vision Assistant). One goal in their approach was to leverage existing models from separate modalities (vision and language) to create a multimodal model. The first of two training steps was "Pre-training for Feature Alignment", where image-caption pairs were used to train a projection layer to map visual features from the image encoder to the vocabulary space of the LLM. This project mimics that exact setup, though it does not continue to the second step "Fine-tuning End-to-End", where the model is trained for more general, visual Q&A tasks.

This project also follows the design of LLaVa by using the CLIP (Contrastive Language-Image Pre-training) vision encoder [2]. CLIP was trained on image-text pairs using a contrastive objective that aligns visual and language representations in a shared embedding space. CLIP is an effective image embedding model for this task because its embeddings capture high-level semantic concepts that have already proven capable of direct alignment, as evidenced by its success on zero-shot learning tasks involving natural language.

Instead of using a high-parameter, LLaMA-based backbone model like LLaVa, Nano-LLaVa adapts Nanochat [3], an open-source, lightweight implementation of a transformer model. Nanochat is designed to enable training "GPT from scratch" without the expensive GPU requirements of larger LLMs. The ability to directly modify Nanochat's GPT implementation and training modules make it suitable for this project's linear projection layer and custom caption task requirements.

III. Methodology

Before any optimization experiments could be conducted, the architecture of Nano-LLaVa needed to be modified for the image-captioning task. Since the goal was to extend an existing LLM, minimal modifications were made to Nanochat's core GPT architecture. A new "vision projection" linear layer was added using Nanochat's native Linear class to map from the 768-dimension CLIP embedding features to the 1024-dimensional vocabulary space of Nanochat. By integrating the visual token injection directly into the existing forward() pass, the model was adapted to recognize these projected features as prefix tokens during the prefill stage.

This architecture was designed to fit into Nanochat's multi-stage training pipeline. Nanochat typically progresses through base training (next token prediction), mid-training (conversational tasks), and supervised fine-tuning (SFT) for specific prompt-response tasks. Because image-captioning is fundamentally a structured prompt-response task, and the strategy was to leverage a pre-trained model, the built-in SFT script was adapted. Since this framework already expects inputs in the typical SFT chat template, the embedded COCO images, captions, and static prompt ("Describe the image") were each formatted as one of these one-shot conversations. This allowed the use of proper attention masking to ensure loss was only calculated on the decoded caption tokens.

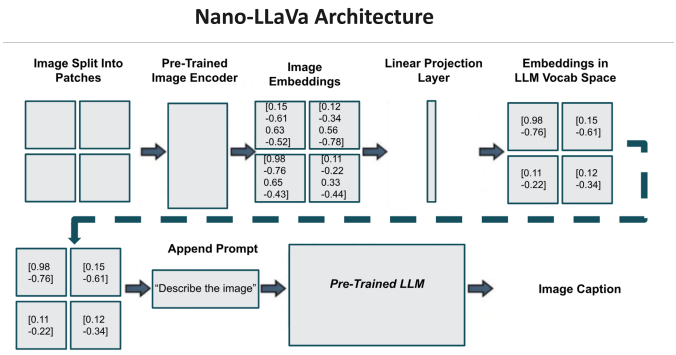

Fig. 1. "Architectural Diagram using Pre-trained Image Encoder and LLM"

Data: The model was trained and evaluated on the COCO Captions 2017 dataset [4], which contains 123,287 training and 5,00 validation images with an average of 5 captions each. The standard practice of duplicating each image to pair it with all of its captions was used to expand both the training and validation dataset. No test split was created because the focus of Nano-LLaVa was to develop an acceptable baseline of caption quality and then focus on training pipeline optimizations. If the goal was generating the absolute highest accuracy captions, more extensive hyperparameter tuning would have been done, along with trying more complex projection layers like MLP's, which LLaVa was shown to benefit from (citation), different pre-trained image embedding models and nanochat weights.

Models: To accommodate the scaled-down resource constraints, Nano-LLaVa uses the CLIP ViT-B/32, which has 32x32 pixel patches rather than the larger CLIP ViT-L/14 model used by LLaVa. Nano-LLaVa also uses a smaller, d20 (20 layer) version of pre-trained Nanochat chat weights rather than the standard, d32 pre-trained weights to ensure the model fits in VRAM of the NVIDIA L4 GPU. Lastly, the all-MiniLM-L6-v2 distilled transformer [5] was utilized to calculate semantic similarity of the generated captions during validation.

Optimization Scope: The optimization scope consists of experimenting with dataloader configurations, transitioning to offline image embeddings, Automatic Mixed Precision (AMP) memory optimization, and tuning inference throughput. Since the goal is to provide the shortest path to image captioning with a minimal training setup, less focus was given to improving image quality beyond a basic baseline during these optimizations. Likewise, these experiments only trained for 30 iterations, enough to provide a warmup and hot measurement period for profiling and representative data loading metrics. The semantic similarity between generated and target image captions was analyzed during BS tuning to ensure the frequency of weight updates did not cause training convergence instability or hurt generalizability. Full training (up to 16.384M tokens) was done during hyperparameter tuning to ensure generalizability of caption quality.

Profiling Tools: Metrics were tracked using the PyTorch Profiler, Tensorboard GPU Summary, custom Weights & Biases instrumentation, and manual torch_profiler.record_function annotations. Experiments were conducted on an NVIDIA L4 Google Cloud VM.

Evaluation Metrics: Hardware efficiency was evaluated via Dataloader Wait Time, H2D Transfer time, Warmup/Hot Time, Total Training Time, GPU Utilization, SM Utilization, and Tokens/Second.

Task Metrics: To ensure the caption quality was beyond a baseline, two main metrics were used. Nanochat's provided a built-in bits per byte (bpb) score, which was used to track loss on the validation set. Although this measures next-token prediction generalizability, semantic similarity between target and generated caption better measures caption quality.

IV. Experiments

A. Tuning DataLoader During Online Image Embeddings

To optimize the data pipeline before adjusting the model itself, initial experiments focused on the PyTorch DataLoader configuration while computing image embeddings online. This involved defining a PyTorch Dataset whose __get_time__ function embeds the image using the CLIP image encoder. Since train time per iteration stabilizes around 20 iterations in based on a baseline run, each experimental run was run for 30 iterations with the measured hot time occurring measuring training tok/sec and training time from iterations 20-30.

Pinned Memory: Pinning memory is a helpful optimization to ensure the memory on the host is at a reliable location and doesn't require paging when the H2D device transfer occurs. Two pairs of experimental runs were conducted to analyze pinned memory's interaction with number of workers. Pinned memory is often used in conjunction with greater number of workers so that not only is the H2D transfer fast, but the CPU has completed the preprocessing of the images fast enough to take advantage of it. Therefore, the expectation is for the total training time speedup from switching to pinned memory to be greatest with pinned memory on. When running with zero workers (nw=0), enabling pinned memory resulted in a slight increase in tokens per second and a decrease in DataLoader wait time. While it decreased "hot" training time by 5.6 seconds, it introduced a significant 19.9-second penalty to warmup time, resulting in a net increase in total training time (+14.2s). However, the tradeoff was deemed worthwhile since reducing hot training time is the primary goal for scaling. When running with the optimal number of workers (4), switching to pinned memory hurt both warmup time and training time.

Overall, pinned memory did not have a signiticant effect at both the baseline and optimal number of workers. With num_workers=0, changing to pinned memory reduced total hot training time from 289.69 to 284.05, a 1.95% decrease in training time. With num_workers=4, changing to pinned memory increased total hot training time from 201.88 to 208.10, a 2.07% increase in training time. Although it decrease average H2D transfer time in both cases .025 to .007 s for num_workers=0 and .041 to .018 s, H2D transfer only occupies .08% of the training time, making these gains insignificant when compared to the dominant factor of the dataloader prefetch.

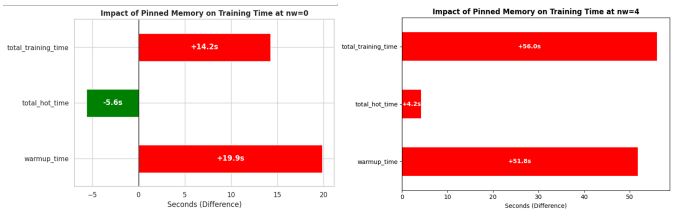

Fig. 2. ”Pinned memory slightly reduces hot training time only with 0 workers”

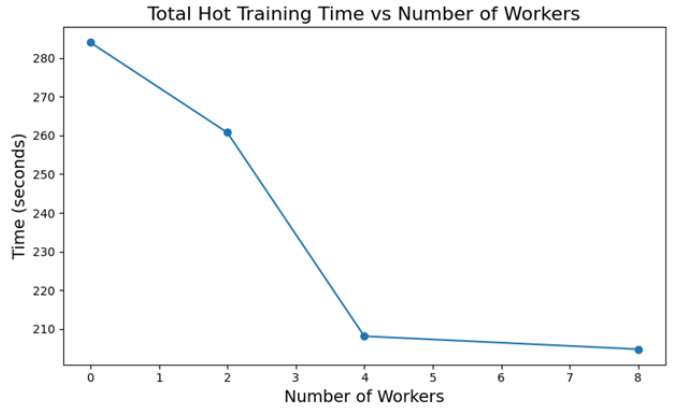

Fig. 3. ”Increasing number of workers beyond number of CPUs shows plateau in training improvement”

Number of Workers: When there is significant preprocessing done by the dataloader when each batch is loaded, increasing the number of workers can help parallelize this task to help parallelize this task to help keep the GPU busy. To tune number of workers, values of 0, 2, 4 and 8 were attempted.

Increasing the number of DataLoader workers from 0 to 4 significantly improved hot training speed, decrease total hot training down from 284.05 to 208.10. However, further increasing number of workers from 4 to 8 only decreased hot training time to 204.73. This concurs with the expected plateau when number of workers increased beyond the number of CPU cores, in this case 4. Beyond that number, there aren't enough CPUs to effectively parallelize workers, causing initialization and overhead of each worker to dominate. When profiling num_workers=0 vs num_workers=4 showed a decrease in CPU Exec as a percentage of total training time from 77.8% to 20.8%. While decreasing CPU Exec is a secondary goal of optimizing the dataloading pipeline, the ultimate goal is to increase the GPU utilization, which only improved from 9.85% to 14.23%. The PyTorch profiler shows that optimizing the number of workers shifted the dominant factor in training time from CPU Exec to “Other”. Manual instrumentation of the dataloader prefetch still shows it taking 13.02 out of the 20.29 seconds of a training iteration, indicating dataloader is still a major bottleneck.

Persistent Workers: Persistent Workers was tested with the optimal number of workers (4). Turned this on slightly increased total hot training time from 208.10 to 214.34 s. The idea of optimizing persistent workers for this pipeline is flawed because there is too much training data to even train for one full epoch. Later experiments showed a plateau in validation scores after 1000 iterations, which does not encompass a full epoch of data. Since the benefit of persistent workers only manifests when the dataloder has fed an entire epoch of training data and maintains the same workers instead of creating new ones, the benefit of persistent workers is never shown.

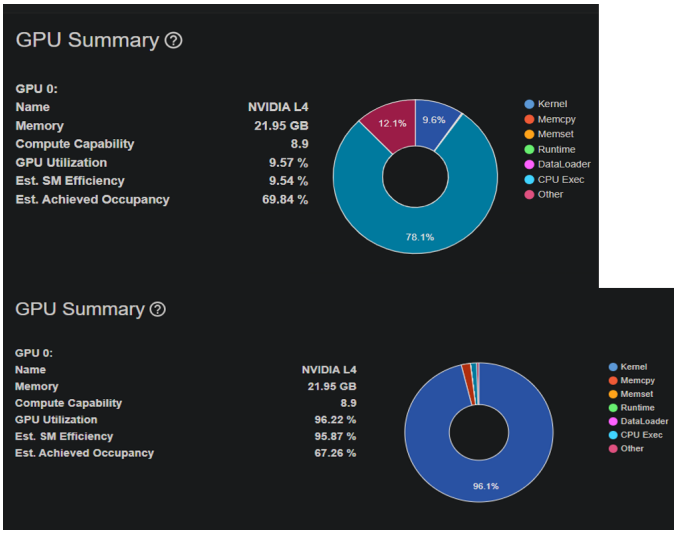

Fig. 4. ”PyTorch Profiler shows dataloading bottleneck from majority time spent in CPU Exec when image embeddings are computed online (top). Switching to offline image embeddings (bottom) dramatically improves Est. Sm Efficiency and kernel usage”

B. Transitioning to Offline Image Embeddings

Although the dataloader was optimized to reduce hot training time from from 284.05 to 208.10 – a 26.7% decrease – profiling still shows dataloader prefetch as taking 64.2% of training time. Since Nano-LLaVa keeps the CLIP vision encoder is always frozen, the same image embeddings are being computed every training run. A natural optimization is to pre-compute the image embeddings offline and have the dataloader load them during training. To validate this approach, the optimal online dataloader setup (num_worker=4, no pinned memory, no persistent workers) was compared to the same setup with the offline image embeddings. The results were that total hot training time decreased from 203.88 to 33.21 s, provided a 6.14x speedup. Profiling confirms that GPU utilization jumps from 13.96% to 96.22%, which is entirely from the reduction in dataloader prefetch time from to 13.32 to .18 s.

Custom Pinned Memory: Without the custom pytorch dataloader for computing the image embeddings, the training pipeline leveraged Nanochat's custom dataloader. Therefore, pinned memory was manually implemented by calling .pin_memory() only after creating the final tensors for the input and targets and making them contiguous. The results showed a negligible decrease in ”hot” training time (from 33.21s to 33.02s), while GPU utilization unexpectedly dropped from 96.22% to 92.54%. Further profiling with manual instrumention revealed an increase in H2D transfer from .007

to .396 s. This reason pinning memory still resulted in a slightly shorter training time was because the backwards pass decreased from 2.040 to 1.766 s. Although the total training time appeared slightly shorter, the profile indicated this was due to a decrease in the backward pass duration (from 2.040s to 1.766s). Because there is no theoretical basis for memory pinning to accelerate the backward pass, this variance was attributed to system noise rather than a true optimization. Given that H2D transfer times were already minimal and the custom pinning implementation negatively impacted both H2D transfer and GPU utilization, it was determined that pinning memory was not optimal. Furthermore, the added complexity of a manual implementation introduced a high risk of bugs.

C. AMP-Driven Batch Optimization

With the data pipeline heavily optimized, experiments shifted toward precision and memory management to maximize device batch sizes. Instead of aiming to reducing training time, this optimization focuses on maximizing throughput by increasing training tokens per second and decreasing VRAM footprint.

Optimizing AMP first is important to reduce the memory footprint and make space for later larger device BS (BS) optimization. To continue leveraging Nanochat's framework, the built-in option for mixed precision was used. Using the optimized dataloading pipeline with offline image embeddings, FP32 was compared to BF16 at the default device BS of 16. This transition resulted in a significant performance gain, increasing average throughput from 6,503 to 9,874 tokens/sec and reducing total hot training time from 50.40s to 33.16s. Peak VRAM memory was reduced from to 9791.44 to 8244.17 MiB and H2D transfer times were nearly halved (from 0.982 to 0.358s). The PyTorch Profiler's Kernel View confirmed the activation of specialized Ampere BF16 Tensor Core kernels. While GPU Utilization showed a minor decrease from 94.84% to 92.19%, this metric is misleading because it only measures the time the GPU is busy rather than how effectively it is processing data. A more useful metric to that actually accounts for the work being done by the tensor cores is Estimated Achieved Efficiency, which improved 47.74% to 68.09%, which more closely aligns with the throughput improvement.

With the slight VRAM memory reduction from using BF16, the next step was to increase the device BS to further increase the amount of work being done per iteration. Keeping AMP constant at BF16, device BS of 8, 16, 32, and 64 were tried. An immediate improvement from using BF16 is that a device BS64 was able to work properly without a GPU Out of Memory error, which occurred with FP32 and BS64. However, this increase in device BS yielded a regression in throughput hot training time. Profiling the optimized BS of 16 vs 64 reveals a tradeoff between the forward and backwards pass. Increasing device BS from 16 to 64 decreases the forward pass from 1.072 to .331 s but increases the backwards pass from 1.768 to 2.991s. One potential explanation is that increasing device batch sizes results in fewer forward and backwards passes per weight update. Therefore, there is less kernel launch overhead for calling the forward pass but more memory bandwidth pressure to store and access the intermediate activations.

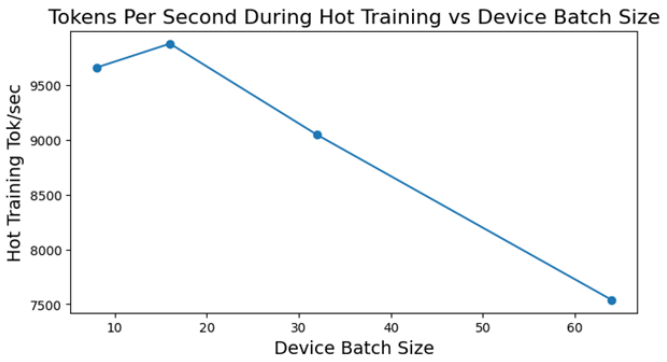

Fig. 5. "Increase Device BS does decreases throughput"

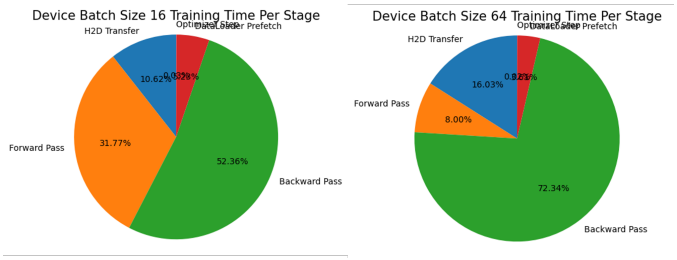

Fig. 6. "With larger device BS, a less time is spent on the forward pass and more time is spend on the backwards pass"

Ultimately, while BF16 allowed for larger batch sizes, these increased batch sizes did not improve throughput and training time because of the bandwidth pressure during the backwards pass.

D. Hyperparameter Tuning

Because the underlying language model and vision encoder were frozen, the primary tunable parameter for model convergence was the learning rate for the single linear projection layer and the total BS (BS), which affects the number of training samples between weight updates. A calibration training run was done for 1,000 iterations, revealing that loss reduction and validation semantic similarity plateau after 15M tokens trained. Since this takes 50 minutes to train, a full hyperparameter grid search with learning rate and total BS was not realistic. Since learning rate is often a dominant factor in convergence, it was tuned first before total BS.

A course-grain search from learning rates 1e-4 to 1e-1 found that the default learning rate of 1e-2 achieved the best average semantic similarity scores and the second lowest Bits per Byte (BPB) on the validation set.

To evaluate the impact of total BS on training stability and model generalizability, total BS was varied along with number of training iterations to maintain a constant total of tokens trained per run. This was necessary because training loss and validation semantic similarity plateau before completing an epoch of training, requiring training time to be determined by iterations (number of total batches). Three total BS configurations were tested: 8,192, 16,384, and 32,768 (500 iterations). A moving average was applied to smooth

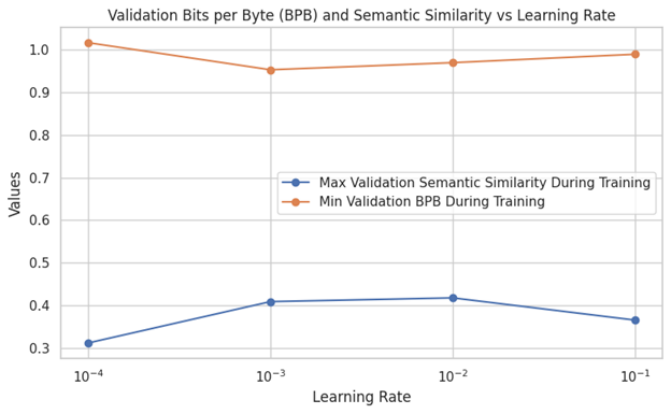

Fig. 7. "The optimal LR was chosen to be 1e-2, which achieves the second lowest Bits per Byte and highest caption quality"

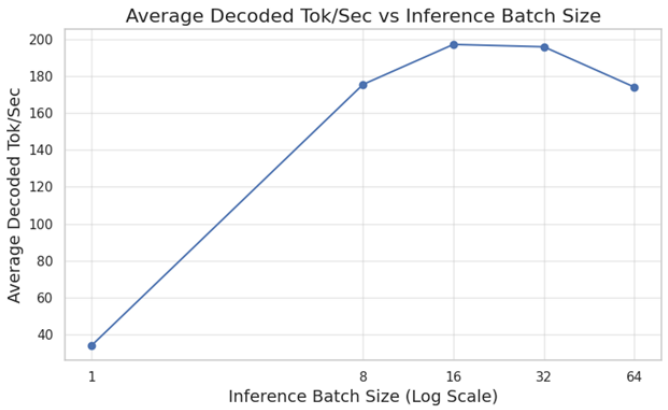

Fig. 9. "Similar to device batch size, increasing inference batch size beyond 16 samples decreases throughput"

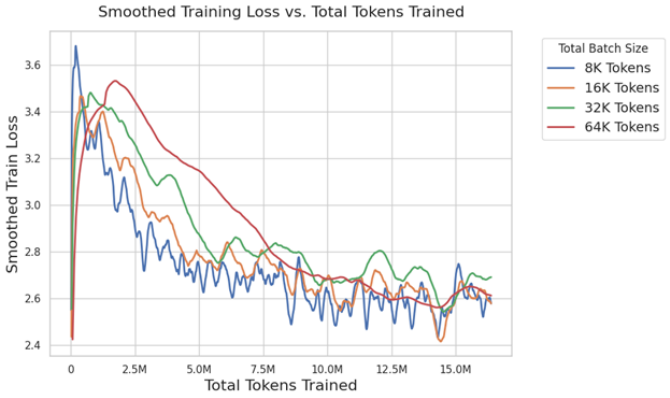

Fig. 8. "Larger total batch sizes increase the amount of samples in each update, making the updates smoother"

the disparities in logging frequencies from different number of iterations trained.

Results indicate that increasing the total BS yields a progressively smoother training loss curve. This observation aligns with the expected theoretical results: larger batch sizes reduce the variance of each weight update, causing a smoother learning curve.

While larger total batch sizes provide optimization stability, smaller batch sizes are often considered because their higher gradient helps the model escape local minima. To evaluate this trade-off, validation performance was measured using average semantic similarity. The results indicated top validation accuracy across all batch sizes did not vary much, with a minimum of 0.386 and maximum of 0.417 across the 4 experiments. Since the smaller batch sizes did not yield a tangible generalization benefit in this specific regime, the larger total BS remains the optimal choice due to its slightly superior training stability.

E. Inference Optimization

The final phase of optimization addressed autoregressive decoding by transitioning from sequential to batched sequence generation. Decoded tokens per second was prioritized as the primary metric because it provides a length-invariant measure of throughput compared to average caption generation time. As the evaluation BS increased, tok/sec improved significantly, plateauing at a BS of 16. Within this optimal window, tok/sec increased from 34.30 to 197.08 for a a 5.75x throughput increase.

Similar to the training BS results, performance gains stagnated beyond BS 16 due to increased memory movement and potential cache spilling. This hardware limit was evidenced by Peak VRAM memory, which remained stable at 8244.17 MiB through BS 16, before climbing as BS scaled further. This suggests that BS 16 represents the hardware's saturation point for this model, where parallel compute gains are eventually neutralized by the overhead of managing larger activations.

V. Discussion

Interpretation: The most significant performance gain across all metrics—throughput, GPU utilization, and total training time—resulted from the transition from online to offline embeddings. Since the CLIP vision encoder is frozen, precomputing the image embeddings is a "zero-cost" optimization. The major takeaway is that optimization efforts should follow the principle of greatest impact; micro-optimizations to a live data pipeline are secondary to architectural changes that eliminate redundant computations. Furthermore, while Automatic Mixed Precision (AMP) successfully increased Estimated Achieved Efficiency and reduced the VRAM footprint, it revealed a clear hardware-specific saturation point. This saturation point was illustrated from training and inference throughput plateauing when device BS was increased beyond 16, which was also the precision point at which Peak GPU Memory began to increase. At this threshold, the benefits of parallelization were likely neutralized by the latency of moving large activations.

Limitations: The primary constraint of this study was that training to convergence required approximately 50 minutes, making it unrealistic to perform a more complete design of experiments. Consequently, many interactions between variables were untested, such as whether reduced precision impacted generalizability.

A primary limitation of this study was the inability to identify the root cause of the performance plateau device

when BS was increased beyond 16. While the PyTorch Profiler revealed an increase in backwards pass time, it has yet to be confirmed that this is due to larger activation and the resulting memory pressure from moving them. The PyTorch Profiler lacked a "Memory View" to confirm cache spilling.

Future Work: While a fundamental goal of this study was to find the minimal training setup needed for image captioning with a consumer-grade GPU, increasing GPU VRAM would allow for larger versions of the CLIP image encoder [2] and a version of pre-trained Nanochat [3] with more layers. A better GPU would also facilitate a more sophisticated projection layer, such as the sequence of Multilayer Perceptrons that was used in the original LLaVa model. Nanochat's build-in KV Caching could also offer performance improvements, although it sidelined in this project because each the learned captions were so short. Further work could also resolve the CUDA driver and library incompatibilities that prevented torch.compile from being supported in Nano-LLaVa. Finally, since this project focused exclusively on image captioning, the model's performance on standard LLM's benchmarks could be tested for degradation. Nano-LLaVa's capabilities could also be extended beyond image-captioning to more general visual question answering (VQA).

VI. Conclusion

This project validated the architectural approach of the LLaVa model [1] while providing a methodology for optimizing the training pipeline to fine-tune an LLM for image-captioning. The training pipeline was improved from GPU-starved to GPU-saturated by offloading image embeddings and AMP-driven BS tuning. Minor hyperparameter tuning also confirmed a stable training setting. Compared to the unoptimized baseline, the final pipeline leveraged online image embeddings and mixed-precision to provide a 82.68% increase in SM utilization and 8.77x speedup in training time. Inference BS optimization provided a 5.75x increase in tokens/sec. These results establish that with an optimized data and memory pipeline, extending an LLM for image captioning can be done with consumer-grade hardware.

References

[1] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning," in *Proc. 37th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2023.
[2] A. Radford et al., "Learning Transferable Visual Models from Natural Language Supervision," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, 2021.
[3] A. Karpathy, "nanoGPT," GitHub repository, 2023. [Online]. Available: https://github.com/karpathy/nanoGPT [Adapted as Nanochat].
[4] T. Y. Lin et al., "Microsoft COCO: Common Objects in Context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.
[5] W. Wang et al., "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020.

amssymb

AI Use Disclosure

☐ No, we did not use any AI tool.
☒ Yes, we used AI assistance as described below.

Tool(s) used: Gemini, Github Copilot

Specific purpose: Gemini was used with planning the project and reducing the scope since I am working solo. It helped me decide on the simpler prefix-token strategy instead of cross-attention. It was also used during writing to re-word and give ideas for transitions between sections, as well as planning the overall structure. It was not used to analyzed results or generate full paragraphs. It was used extensively setup and debugging of pytorch profiler, uv, GCP, LateX, Tensorboard, and Wandb.

Copilot was used for explaining the existing Nanochat repo and for generating boilerplate code, such as the code for loading images from CLIP or for most of the matplotlib plot generation. For plots, I would specificy the wandb runs and metrics to plot as well as which type of plot. Copilot code attributes are visible in the commit history.

How we verified correctness: After AI suggested the simpler prefix-token strategy, I read the LLaVa paper myself to see if it matched with my intended project. For code generation, I ran multiple times after each generated commit to reduce probability of added bugs.

By submitting this project, the team confirms that the analysis, interpretations, and conclusions are our own, and that any AI assistance is fully disclosed above. The same disclosure block appears as an appendix in the final report.